CSCE 221 Cover Page

Please list all sources in the table below including web pages which you used to solve or implement the current homework. If you fail to cite sources you can get a lower number of points or even zero, read more Aggie Honor System Office https://aggiehonor.tamu.edu/

If you are caught abusing LLMs or GenAI such as ChatGPT, Claude, etc... or services like Chegg or CourseHero to come up with solutions and or code that you did not create yourself, we are obligated to (and will) file a report to the Aggie Honor System Office with no exceptions. For a list of more details and potential sanctions read more on the honor code site here.

Name	Justus Languell
UIN	135007144
Email address	justus@tamu.edu

Cite your sources using the table below. Interactions with TAs and resources presented in lecture do not have to be cited.

People	1. None
Webpages	1. tex.stackexchange.com 2. stackoverflow.com
Printed Materials	1. None
Other Sources	1. None

Homework 2

Due date is on the Canvas calendar

Typeset your solutions to the homework problems preferably in LaTeX or LyX. See the class webpage for information about their installation and tutorials.

1. (15 points) Provided two **sorted** lists, `l1` and `l2`, write a function in C++ to *efficiently*[1] compute the intersection `l3` = `l1` $\cap$ `l2` **using only the basic Standard Template Library (STL) list operations**. The lists may be empty or contain a different number of elements e.g. $|\texttt{l1}| \neq |\texttt{l2}|$ You may assume `l1` and `l2` will not contain duplicate elements.

   Examples (all set members are list node):

   - $\{1, 2, 3, 4\} \cap \{2, 3\} = \{2, 3\}$
   - $\emptyset \cap \{2, 3\} = \emptyset$
   - $\{2, 9, 14\} \cap \{1, 7, 15\} = \emptyset$

   (a) Complete the function below. Do not use any routines from the algorithm header file.

```cpp
#include <list>

std::list<int> intersection(const std::list<int>& l1,
                            const std::list<int>& l2) {
    std::list<int> l3;
    for (auto i1 = l1.begin(), i2 = l2.begin();
         i1 != l1.end() && i2 != l2.end(); )
    {
        if (*i1 < *i2) i1++;
        else if (*i2 < *i1) i2++;
        else {
            l3.push_back(*i1++);
            i2++;
        }
    }
    return l3;
}
```

   (b) Verify that your implementation works properly by writing two test cases. Please provide the cases and terminal output of your tests below.

   (1)

```
l1 = { 1 2 3 4 }
l2 = { 2 3 }
l3 = { 2 3 }
```

[1] Refers to the best correctly performing algorithm on grounds of big-O time complexity; for this question, you may assume that efficient means better than quadratic.

(2)

```
l1 = { 1 3 5 6 17 23 }
l2 = { 1 4 6 17 18 23 29 }
l3 = { 1 6 17 23 }
```

(c) What is the running time function for your algorithm? Provide a running time function and a big-O bound. Justify.

Given n and m, the length of l1 and l2 respectively, the total runtime is $O(n+m)$, because the worst case is that the loop traversal iterates all the way to the end of one of the lists, then as to iterate all the way to the back of the other.

2. (15 points) Write a C++ recursive function that counts the number of nodes in a singly linked list. Do not modify the list.

Examples:

- `count_nodes`$((2) \to (4) \to (3) \to$ `nullptr`$) = 3$
- `count_nodes(nullptr)` $= 0$

(a) Complete the function below:

```
template<typename T>
struct Node {
  Node* next;
  T obj;

  Node(T obj, Node* next = nullptr)
    : obj(obj), next(next)
  { }
};

template<typename T>
int count_nodes(Node<T>* node) {
    if (node == nullptr) return 0;
    return 1 + count_nodes(node->next);
}
```

(b) Verify that your implementation works properly by writing two test cases for the function you completed in part 2a. Please provide the cases and terminal output of your tests below.

(1)

```
list = { 5 4 3 2 1 }
# nodes = 5
```

(2)

```
list = { 14 6 17 54 10 8 12 14 }
# nodes = 8
```

(c) Write a recurrence relation that represents your algorithm.

$$T(n) = T(n-1) + O(1)$$

(d) Solve the recurrence relation using the iterative or recursive tree method to obtain the running time of the algorithm in Big-O notation. Show your process.

$$T(n) = T(n-1) + 1$$
$$T(n-1) = T(n-2) + 1$$
$$T(n-2) = T(n-3) + 1$$
$$\vdots$$
$$T(1) = T(0) + 1 = 0 + 1 = 1$$

so,

$$T(1) = 1$$
$$T(2) = T(1) + 1 = 2$$
$$\vdots$$
$$T(n-1) = T(n-2) + 1 = (n-2) + 1 = n - 1$$
$$T(n) = T(n-1) + 1 = (n-1) + 1 = n$$

therefore,

$$T(n) = n$$
$$O(n)$$

3. (15 points) Write a C++ recursive function that finds the maximum value in an array (or vector) of integers *without* using any loops. You may assume the array will always contain at least one integer. Do not modify the array.

(a) Complete the function below:

```
#include <vector>

int find_max_value(std::vector<int> v, int m = INT32_MIN, int i =
    0) {
     if (i == v.size()) return m;
     if (v[i] > m) m = v[i];
     return find_max_value(v, m, i + 1);
}
```

(b) Verify that your implementation works properly by writing two test cases. Please provide the cases and terminal output of your tests below.

(1)

```
array = { 8 3 4 19 34 2 34 8 10 5 11 8 }
max value = 34
```

(2)

```
array = { 34 7 23 32 5 62 78 45 89 12 }
max value = 89
```

(c) Write a recurrence relation that represents your algorithm.
Same as last,

$$T(n) = T(n-1) + O(1)$$

(d) Solve the recurrence relation and obtain the running time of the algorithm in Big-O notation. Show your process.

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ T(n-1) &= T(n-2) + 1 \\ T(n-2) &= T(n-3) + 1 \\ &\vdots \\ T(1) &= T(0) + 1 = 0 + 1 = 1 \end{aligned}$$

so,

$$\begin{aligned} T(1) &= 1 \\ T(2) &= T(1) + 1 = 2 \\ &\vdots \\ T(n-1) &= T(n-2) + 1 = (n-2) + 1 = n - 1 \\ T(n) &= T(n-1) + 1 = (n-1) + 1 = n \end{aligned}$$

therefore,

$$\begin{aligned} T(n) &= n \\ O(n) \end{aligned}$$

4. (15 points) What is the best, worst and average running time of quick sort algorithm?

(a) Provide recurrence relations. For the average case, you may assume that quick sort partitions the input into two halves proportional to c and $1-c$ on each iteration.

- best: $T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + O(n) = 2T(\frac{n}{2}) + O(n)$
- average: $T(n) = T(c \cdot n) + T((1-c) \cdot n) + O(n)$
- worst: $T(n) = T(n-1) + T(0) + O(n) = T(n-1) + O(n)$

(b) Solve each recurrence relation you provided in part 4a. Show your process.

- best:

$$T(n) = 2T(\frac{n}{2}) + O(n)$$
$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + O(\frac{n}{2})$$

where i is the level index:

$$T(n) = 2^i T(\frac{n}{2^i}) + in$$

$\frac{n}{2^i} = 1 \rightarrow 2^i = n \rightarrow i = log(n)$, therefore the order is $O(n \cdot log(n))$.

- average:

$$T(n) = T(c \cdot n) + T((1-c) \cdot n) + O(n)$$

At some point c and $1-c$ approach a the idealized partition, as $c-(1-c) = 2c-1$ approaches $\frac{n}{2}$.
Therefore, it approaches the ideal best case, and evaluates to order $O(n \cdot log(n))$

- worst:

$$T(n) = T(n-1) + O(n)$$
$$T(n-1) = T(n-2) + O(n)$$
$$\vdots$$
$$T(1) = T(0) + O(n) = O(n)$$

therefore, $T(n) = O(n) \cdot O(n)$, so the order is $O(n^2)$.

(c) Provide an arrangement of an input array of size $n = 7$ which results in each case. Assume the first item is always chosen as the pivot for each iteration.

- best: 4, 2, 3, 1, 5, 6, 7
- average: 3, 2, 6, 5, 4, 1, 7
- worst: 7, 6, 5, 4, 3, 2, 1

5. (15 points) Write a C++ recursive function that counts the total number of nodes with two children in a binary tree (do not count nodes with one or zero children).

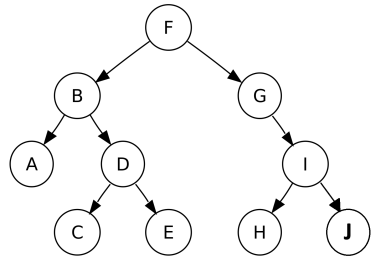

Figure 1: Calling `count_filled_nodes` on the root node F returns `4`

(a) Complete the function below. The function will be called with the root node (e.g. `count_filled_nodes(root)`). The tree may be empty. Do not modify the tree.

```
#include <vector>

template<typename T>
struct Node {
  Node<T>* left;
  Node<T>* right;
  T obj;

  Node(T o, Node<T>* l = nullptr, Node<T>* r = nullptr)
    : obj(o), left(l), right(r)
  { }
};

template<typename T>
int count_filled_nodes(const Node<T>* node) {
    if (node == nullptr || node->left == nullptr
            || node->right == nullptr) return 0;
    return 1 + count_filled_nodes(node->left)
            + count_filled_nodes(node->right);
}
```

(b) Use big-O notation to classify your algorithm. Show how you arrived at your answer using either a recurrence relation or logical reasoning.

In order to count every filled node, the algorithm must visit every filled node. If n is the total number of nodes, the total number of filled nodes is some subset of the set of nodes, of which the size can be described as $m = kn$, where k is some constant $0 < k < 1$. Therefore, the time complexity can be evaluated as $O(m) = O(kn) = O(n)$, so the algorithm runs in the order $O(n)$.

6. (15 points) Each of the following statements about red-black trees is either true or false. For each statement, state whether they are true or false. If you believe that a statement is true, provide a logical justification for your position. If you believe that a statement is false, provide a counterexample that disproves the statement.

   (a) A subtree of a red-black tree can be recolored to a valid red-black tree. Recoloring means you can change whether a node is black or red but cannot perform any rotations/reorientations.

   False. Not always, because if you have a case where your subtree yields a hierarchy black–red–red, then you can switch colors to fix it.

   (b) The sibling of an external/NULL node is either external/NULL or red.

   True. Null leafs are implicitly black, so if the sibling is not null and black, then there exists a miscoloring, so the sibling must be red.

   (c) There is a unique 2-4 tree associated with a given red-black tree (i.e. there exists only 1 equivalent 2-4 tree for any given red-black tree).

   True. A red node is like the third node in the 2-4 tree.

   (d) There is a unique red-black tree associated with a given 2-4 tree (i.e. there exists only 1 equivalent red-black tree for any given 2-4 tree).

   False. Any 2-item node on a 2-4 tree can be turned into a number of different combinations of red-black structure.

7. (10 points) Show the skip list after performing the each following series of operations: `erase(38)`, `insert(48)`, `insert(24)`, `erase(42)`. Your answer should have 5 tables including the original table. If you would like to draw the skip list by hand or represent it in some other way, feel free to do so as long as your representation is clear and easily legible. Additionally, provide the recorded coin flips for each insertion.

$-\infty$	—	—	—	—	—	$+\infty$
$-\infty$	—	17	—	—	—	$+\infty$
$-\infty$	—	17	—	—	42	$+\infty$
$-\infty$	—	17	—	—	42	$+\infty$
$-\infty$	12	17	—	38	42	$+\infty$
$-\infty$	12	17	20	38	42	$+\infty$

(1) erase(38):

```
-inf -  -   -   -   +inf
-inf -  17  -   -   +inf
-inf -  17  -   42  +inf
-inf -  17  -   42  +inf
-inf 12 17  -   42  +inf
-inf 12 17  20  42  +inf
```

(2) insert(48): I coin flipped heads then tails, so height is 2.

```
-inf -  -   -   -   -   +inf
-inf -  17  -   -   -   +inf
-inf -  17  -   42  -   +inf
-inf -  17  -   42  -   +inf
-inf 12 17  -   42  48  +inf
-inf 12 17  20  42  48  +inf
```

(3) insert(24): I coin flipped heads then heads then tails, so height is 3.

```
-inf -  -   -   -   -   -   +inf
-inf -  17  -   -   -   -   +inf
-inf -  17  -   -   42  -   +inf
-inf -  17  -   24  42  -   +inf
-inf 12 17  -   24  42  48  +inf
-inf 12 17  20  24  42  48  +inf
```

(4) erase(48):

```
-inf -  -   -   -   -   +inf
-inf -  17  -   -   -   +inf
-inf -  17  -   -   42  +inf
-inf -  17  -   24  42  +inf
-inf 12 17  -   24  42  +inf
-inf 12 17  20  24  42  +inf
```